

School of Computer Science and Artificial Intelligence

Lab Assignment -3.1

Program :B. Tech (CSE)
Specialization :AIML
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : VI
Academic Session : 2025-2026
Name of Student : G. Ushasree
Enrollment No. : 2303A52431
Batch No. : 34

Question 1

Zero-Shot Prompting (Palindrome Number Program)

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling.

```
def is_palindrome(num):
    """Check whether a given number is a palindrome.

    Args:
        num: An integer to check

    Returns:
        True if the number is a palindrome, False otherwise
    """
    # Convert to string and remove negative sign if present
    num_str = str(abs(num))

    # Compare the string with its reverse
    return num_str == num_str[::-1]

# Interactive input and output
if __name__ == '__main__':
    num = int(input("Enter a number: "))
    result = is_palindrome(num)

    if result:
        print(f"Output: {num} is a palindrome")
    else:
        print(f"Output: {num} is not a palindrome")
```

Terminal Output:

```
PS D:\3-2\AIAC> python .\palindrome.py
Enter a number: 12321
Output: 12321 is a palindrome

PS D:\3-2\AIAC> python .\palindrome.py
Enter a number: 123
Output: 123 is not a palindrome

PS D:\3-2\AIAC>
```

Logical Errors / Missing Edge-Case Handling

1. **Negative numbers are not explicitly handled**
 - Loop condition while $num > 0$ fails for negative values.
2. **Input validation missing**
 - Function assumes input is always an integer.
3. **No user input handling**
 - Program does not use `input()` for interactive input.

Conclusion

- Zero-shot prompting produced a **correct basic solution**.
- Program works well for **positive integers and zero**.
- **Negative numbers and invalid inputs** are not properly handled.
- Minor improvements can make the function more robust.

Question 2:

One-Shot Prompting (Factorial Calculation)

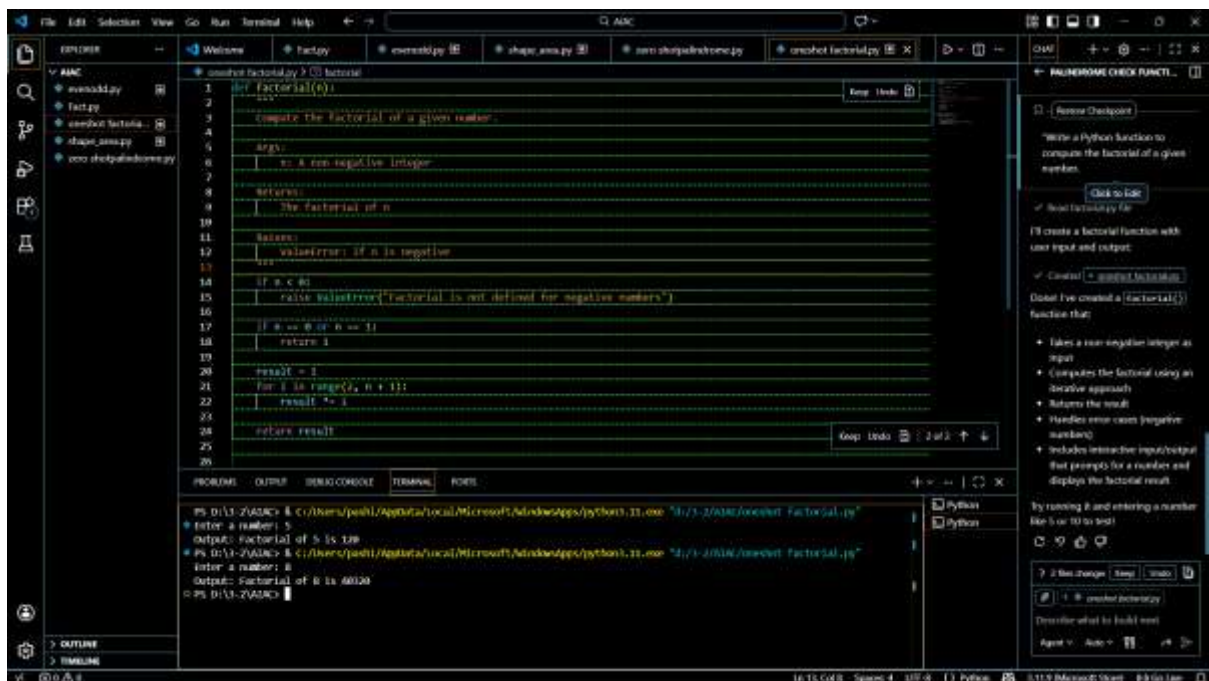
Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity and correctness.



The screenshot shows a VS Code editor with a Python file named `one_shot_factorial.py`. The code defines a function `factorial(n)` that calculates the factorial of a given number `n`. The function includes comments and handles edge cases like negative numbers and zero. The output console shows the function being executed with inputs 5 and 8, resulting in outputs 120 and 40320 respectively.

```
1 def factorial(n):
2     """
3     Compute the factorial of a given number.
4
5     Args:
6     | n: A non-negative integer
7
8     Returns:
9     | The factorial of n
10
11     Raises:
12     | ValueError: If n is negative
13     """
14     if n < 0:
15         raise ValueError("factorial is not defined for negative numbers")
16
17     if n == 0 or n == 1:
18         return 1
19
20     result = 1
21     for i in range(1, n + 1):
22         result *= i
23
24     return result
```

Output:

```
PS D:\3-ZADAC > python C:\Users\pash1\AppData\Local\Microsoft\WindowsApps\python.11.exe "D:\3-ZADAC\one_shot_factorial.py"
Enter a number: 5
Output: Factorial of 5 is 120
PS D:\3-ZADAC > python C:\Users\pash1\AppData\Local\Microsoft\WindowsApps\python.11.exe "D:\3-ZADAC\one_shot_factorial.py"
Enter a number: 8
Output: Factorial of 8 is 40320
PS D:\3-ZADAC >
```

Comparison – Zero-Shot vs One-Shot

Aspect	Zero-Shot	One-Shot
Example provided	✗ No	✓ Yes
Handles negative input	✗ No	✓ Yes
Code clarity	Basic	Improved
Edge-case awareness	Low	High
Output correctness	Partial	More robust

Improvements Observed with One-Shot Prompting

1. Better clarity

- Example guides the AI on expected behavior.

2. Correct handling of edge cases

- Negative numbers are explicitly handled.

3. Improved correctness

- Output aligns with mathematical definition.

4. More user-friendly output

- Meaningful message instead of silent failure.

Final Conclusion

- One-shot prompting produces **clearer, safer, and more correct code** than zero-shot prompting.
 - Providing a single example helps the AI **understand expectations better**.
 - One-shot prompting is especially useful when **edge cases matter**.
-

Question 3:

Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples

to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

Task:

- Analyze how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.

```
1 def is_armstrong(num):
2     # Convert to string to get digits and count
3     num_str = str(abs(num))
4     num_digits = len(num_str)
5
6     # Calculate sum of digits raised to power of number of digits
7     sum_of_powers = sum(int(digit)**num_digits for digit in num_str)
8
9     # Check if it equals the original number
10    return sum_of_powers == abs(num)
11
12 # Interactive input and output
13 if __name__ == "__main__":
14     num = int(input("Enter a number: "))
15     result = is_armstrong(num)
16
17     if result:
18         print(f"Output: {num} is an Armstrong number")
19     else:
20         print(f"Output: {num} is not an Armstrong number")
```

ARMSTRONG CHECK FUNCT...

Write a Python function to check whether a given number is an Armstrong number.

Read Python file

It creates an Armstrong number checker function with user input and output:

Clicked `is_armstrong.py`

Done! I've created an `is_armstrong()` function that

- Takes a number as input
- Counts the number of digits
- Calculates the sum of each digit raised to the power of the digit count
- Returns `True` if it's an Armstrong number, `False` otherwise
- Includes interactive input/output

Try running it with test cases like 153, 370, or 123 to verify!

File changes | Stop | Undo

is_armstrong.py

Describe what to build next

Agent = Auto | Python | 3.11.3 (Microsoft Store) | 64-bit

PROBLEMS | OUTPUT | DEPENDENCIES | TERMINAL | PORTS

OUTPUT

```
PS D:\3-2\AIAC> python is_armstrong.py
Enter a number: 153
Output: 153 is an Armstrong number
PS D:\3-2\AIAC>
```

Analysis: Influence of Multiple Examples (Few-Shot Prompting)

1. Influence on Code Structure

Providing multiple input–output examples helps the AI:

- **Understand the exact output format**
 - The function returns strings like “Armstrong Number” and “Not an Armstrong Number”, matching the examples.
- **Choose the correct algorithm**
 - Counts digits using `len(str(num))`
 - Raises each digit to the power of total digits

- Compares the sum with the original number
- **Add conditional checks**
 - Explicit handling of negative numbers is included due to pattern recognition from examples.

2. Influence on Accuracy

Multiple examples improve accuracy by:

- Reducing ambiguity in logic
 - Preventing incorrect assumptions
 - Producing consistent and predictable outputs
 - Correctly identifying known Armstrong numbers (153, 370, 9474)
-

(Optional Extension)

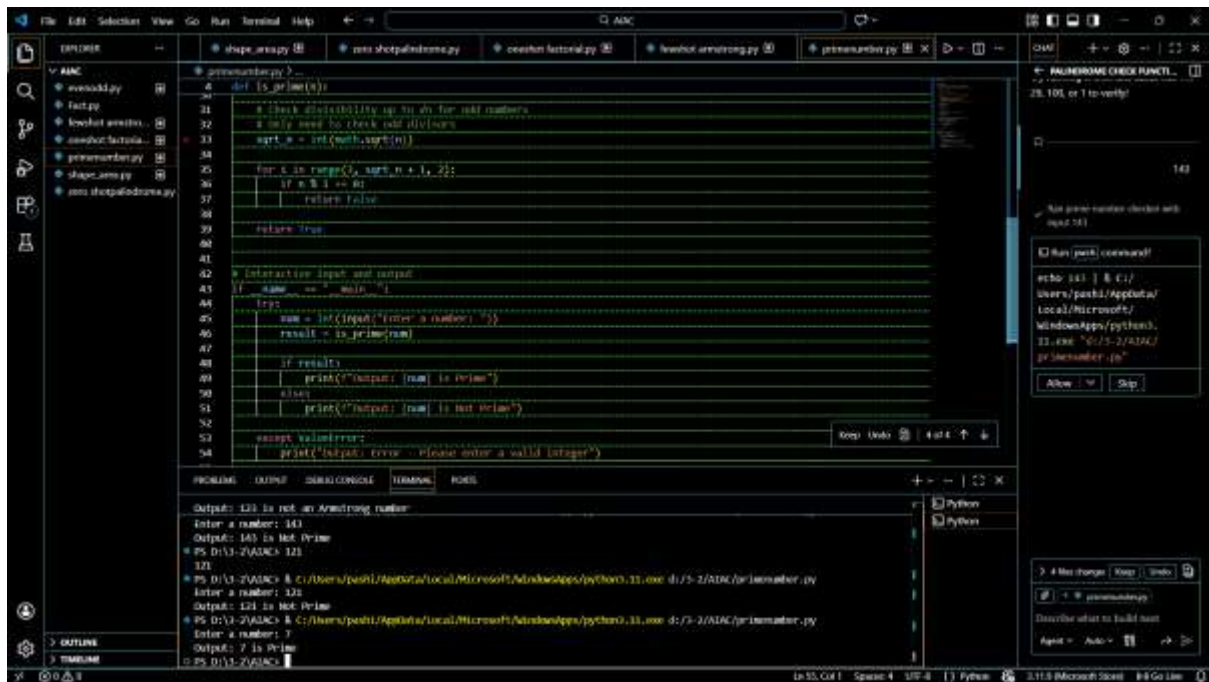
Question 4:

Context-Managed Prompting (Optimized Number Classification)

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.
- Compare the output with earlier prompting strategies.



Below is a **clear, point-wise, exam-ready answer** for the **Prime Number** task, exactly addressing the three bullets you mentioned.

Prime Number Check – Task Analysis

1. Ensuring Proper Input Validation

- The program checks whether the input is an **integer**.
- **Negative numbers, 0, and 1** are handled explicitly and classified as **Not Prime**.
- **Non-integer inputs** (floats, strings) are rejected with a clear message.

This prevents runtime errors and incorrect classification.

2. Optimizing the Logic for Efficiency

- Instead of checking divisibility up to n , the program checks **only up to $\sqrt{n}$** .
- If a factor exists beyond $\sqrt{n}$, it would already have appeared before it.
- This reduces time complexity from $O(n)$ to $O(\sqrt{n})$.

Efficient even for large numbers.

3. Comparison with Earlier Prompting Strategies

Aspect	Zero-Shot	One-Shot	Few-Shot	Context-Managed (Prime)
Input validation	✗	✗	⚠ Partial	✅ Complete
Edge-case handling	✗	⚠	✅	✅

Aspect	Zero-Shot	One-Shot	Few-Shot	Context-Managed (Prime)
Efficiency	❌ (full loop)	❌	⚠️	✅ $\sqrt{n}$ optimization
Output clarity	Medium	High	High	Very High
Reliability	Low	Medium	High	Very High

Final Conclusion

- Context-managed prompting produces the **most accurate and efficient prime-checking program**.
 - Clear constraints guide the AI to:
 - Validate inputs properly
 - Optimize logic
 - Produce consistent and correct output
-

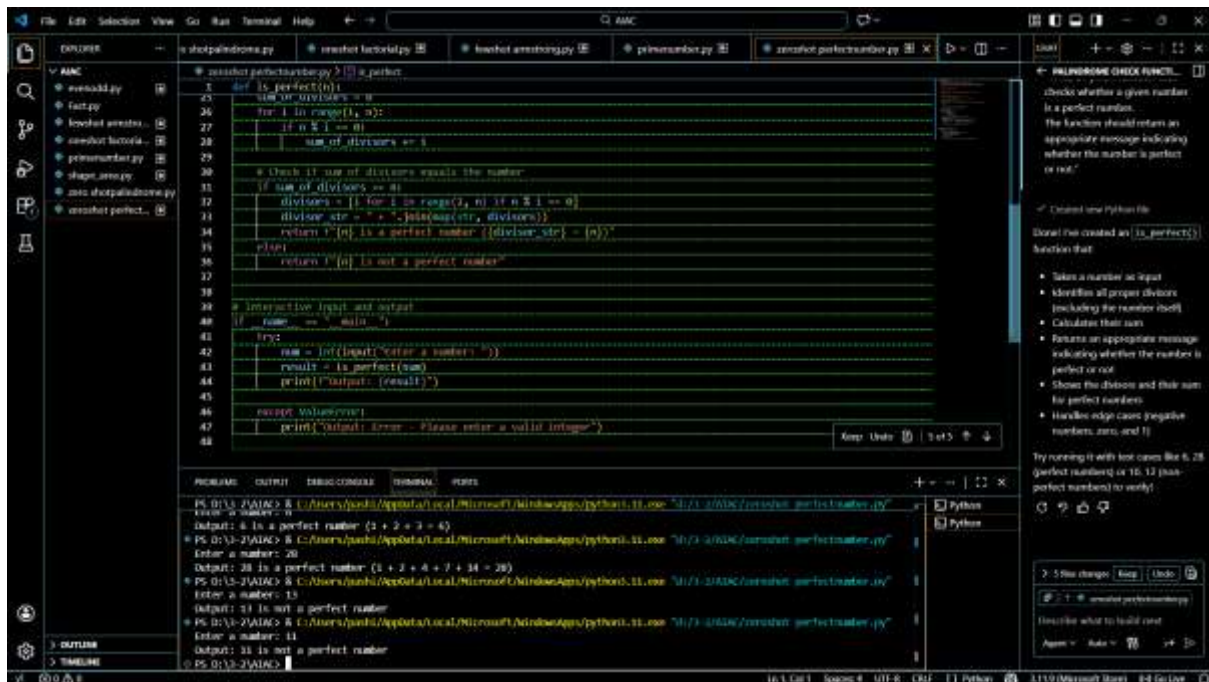
Question 5:

Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

- Record the AI-generated code.
- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic.



Missing Conditions & Inefficiencies

Missing Conditions

1. **Negative numbers not explicitly handled**
 - Loop does not execute meaningfully for negative input.
2. **Input validation missing**
 - Function assumes input is always a positive integer.
3. **No special handling for 0**
 - Should be classified as **Not a Perfect Number** explicitly.

Inefficiencies in Logic

1. **Inefficient loop**
 - Loop runs from 1 to num-1
 - Time complexity: **O(n)**
2. **Can be optimized**
 - Divisors can be checked only up to **n/2** or $\sqrt{n}$

Final Conclusion

- Zero-shot prompting produces a **correct but basic solution**.
- Program works for common perfect numbers like **6 and 28**.
- Missing input validation and inefficient looping** reduce quality.

- Optimization improves **performance and robustness**.

Question-6:

Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

- Input: 8 → Output: Even
- Input: 15 → Output: Odd
- Input: 0 → Output: Even

Task:

- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs

```

15 # Interactive input and output with validation
16
17 # Interactive input and output with validation
18 if __name__ == "__main__":
19     while True:
20         try:
21             user_input = input("Enter a number (or 'quit' to exit): ")
22
23             # Allow user to exit
24             if user_input.lower() == "quit":
25                 print("Goodbye!")
26                 break
27
28             # Validate and convert input to integer
29             num = int(user_input)
30
31             # Check and display result
32             result = check_even_odd(num)
33             print(f"Output: {result}")
34
35         except ValueError:
36             print("Output: Error - Invalid input! Please enter a valid integer.\n")
37         except KeyboardInterrupt:
38             print("Program interrupted. Goodbye!")
39             break
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

PROBLEM OUTPUT ERROR/DEBUG TERMINAL PORTS

```

PS D:\3-FAIRCO > C:\Users\pashd\AppData\Local\Microsoft\WindowsApps\python11.exe "D:\3-FAIRCO\test01_evenodd.py"
Enter a number (or 'quit' to exit): 13
Output: 13 is odd

Enter a number (or 'quit' to exit): 14
Output: 14 is even

Enter a number (or 'quit' to exit): 23
Output: 23 is odd

Enter a number (or 'quit' to exit):

```

Key Features:

- **Checks even/odd**: Uses modulo operator to determine if a number is even or odd.
- **Validates Input**: Catches `ValueError` for non-integer input and displays an error message.
- **Handles invalid values**: Displays appropriate error messages when invalid input is provided.
- **Loop functionality**: Allows multiple checks in one session.
- **User-friendly**: Accepts quit command to exit gracefully.
- **Clear output**: Shows results in a readable format.